

INTRODUCERE ÎN STUDIUL MICROSISTEMELOR ELECTRONICE

1. Obiectul lucrării

Lucrarea își propune o introducere în studiul sistemelor cu microprocesorul Z80. Sunt descrise componentele microsistemului și rolul acestora. Se introduc comenzile programului monitor, prin intermediul căruia se acționează în mod direct asupra microsistemelor existente în laborator. Este prezentat ansamblul de programe tip mediu de dezvoltare IAR Systems Embedded Workbench și comenzile de bază pentru depanarea și simularea programelor în limbaj de asamblare sau realizarea de programe în limbaj de nivel înalt. Lucrarea propune și o scurtă recapitulare a sistemelor de numerație binar și hexazecimal. În final sunt prezentați regiștrii de lucru pentru microprocesorul Z-80.

2. Breviar teoretic

2.1. Elemente generale

Un microsistem de calcul este compus din următoarele elemente:

1. Unitatea centrală (UC), cu rol de comandă și control a întregii structuri. La microsistemele de calcul aceasta este realizată de obicei în jurul unui microprocesor. El poate forma unitatea centrală singur sau împreună cu alte circuite.

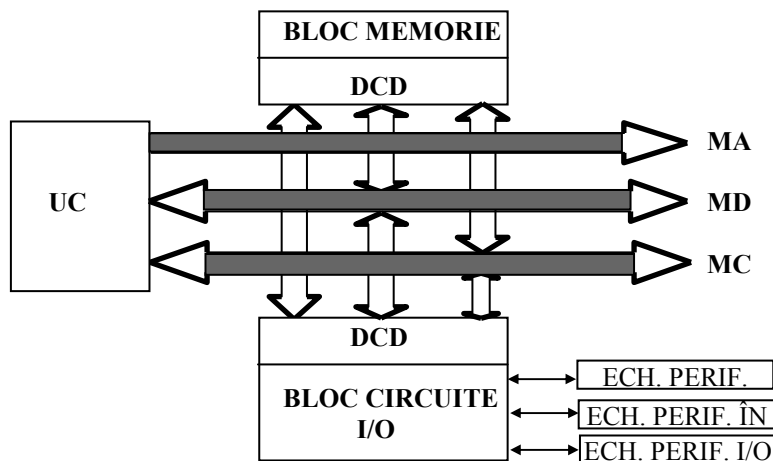
2. Circuitele de memorie, respectiv circuitele de decodificare și selecție aferente:

- memorii ROM - nevolatile - pentru păstrarea programelor rezidente permanent în microsistem. Tehnologic acestea pot fi memorii ROM – OTP (one time programming) EPROM, EEPROM, FLASH, etc.;

- memorii RAM pentru păstrarea programelor utilizator; la dispariția tensiunii de alimentare informația stocată în acest tip de memorii se pierde. Memoriile RAM pot fi de tipul Static RAM (SRAM) sau Dinamic RAM (DRAM).

3. Circuitele specializate de intrare - ieșire (input - output, I/O), respectiv circuitele de decodificare și selecție aferente; prin intermediul acestor circuite (cuploare) sunt gestionate echipamentele periferice.

4. Echipamentele periferice; prin intermediul echipamentelor periferice, un microsistem de calcul se interfațează cu mediul exterior, inclusiv cu operatorul uman.



5. Magistralele informaționale. Elementele microsistemului comunică prin intermediul unor magistrale. O magistrală este constituită dintr-un mănunchi de fire pe care se vehiculează semnale având aceleași semnificații logice. În mod uzual sunt trei categorii de magistrale:

- **magistrala de adrese (MA)** conține informații emise de unitatea centrală în vederea adresării locațiilor de memorie și a circuitelor de intrare - ieșire. Prescurtat, acest tip de informații se numesc adrese. În anumite situații particulare, rolul de generare al semnalelor de adresă nu mai revine unității centrale. De exemplu, în cazul folosirii circuitelor DMA (direct memory access), rolul generării adreselor revine acestor circuite.

- **magistrala de date (MB)** conține liniile fizice pe care se vehiculează informațiile propriu-zise. Prescurtat, acest tip de informații se numesc date. Magistrala de date este bidirecțională. Sensul de parcurgere al magistralei poate fi de la procesor la circuitele I/O sau memoriei, atunci când unitatea centrală efectuează operații de scriere, sau de la memorie sau circuitele I/O către procesor, în situația efectuării operațiilor de citire de către UC. În situații particulare, datele pot să circule între circuitele de intrare - ieșire și memorii, fără a se mai trece prin unitatea centrală (cazul transferurilor DMA).

- **magistrala de comenzi (MC)** conține semnale prin care procesorul se sincronizează în funcționare, cu elementele externe. Ea nu este o magistrală bidirecțională propriu-zisă, ci conține semnale care sunt generate de unitatea centrală și, respectiv, semnale care sunt recepționate de aceasta. Prin intermediul semnalelor din cadrul acestei magistrale se gestionează transferul de date între unitatea centrală și resursele microsistemului (memorii și circuite de I/O).

Numărul de linii din cadrul magistralei de adrese indică numărul de locații de memorie ce pot fi adresate de către procesor. De exemplu, dacă numărul de linii este 14, numărul locațiilor adresate este $2^{14} = 16k$, unde $1k = 2^{10}$. Pentru alte

Îndrumar de laborator 1

numere de linii în cadrul magistralei de adrese se obțin următoarele capacități de adresare:

16 linii	$2^{16} = 64k$
20 linii	$2^{20} = 1M$
24 linii	$2^{24} = 16M$

De remarcat faptul că informația de adresă se aplică atât circuitelor de memorie cât și celor de I/O. Selecția propriu-zisă a unuia din cele două tipuri de circuite revine unor semnale din cadrul magistralei de comenzi.

Mărimea magistralei de date (numărul de linii cuprinse) constituie o caracteristică a microsistemului, indicând mărimea operanzilor care sunt transferați între procesor și memorie sau circuitele I/O. De obicei, acest număr indică și dimensiunea în biți a operanzilor procesați direct de unitatea centrală, practic puterea de calcul a procesorului.

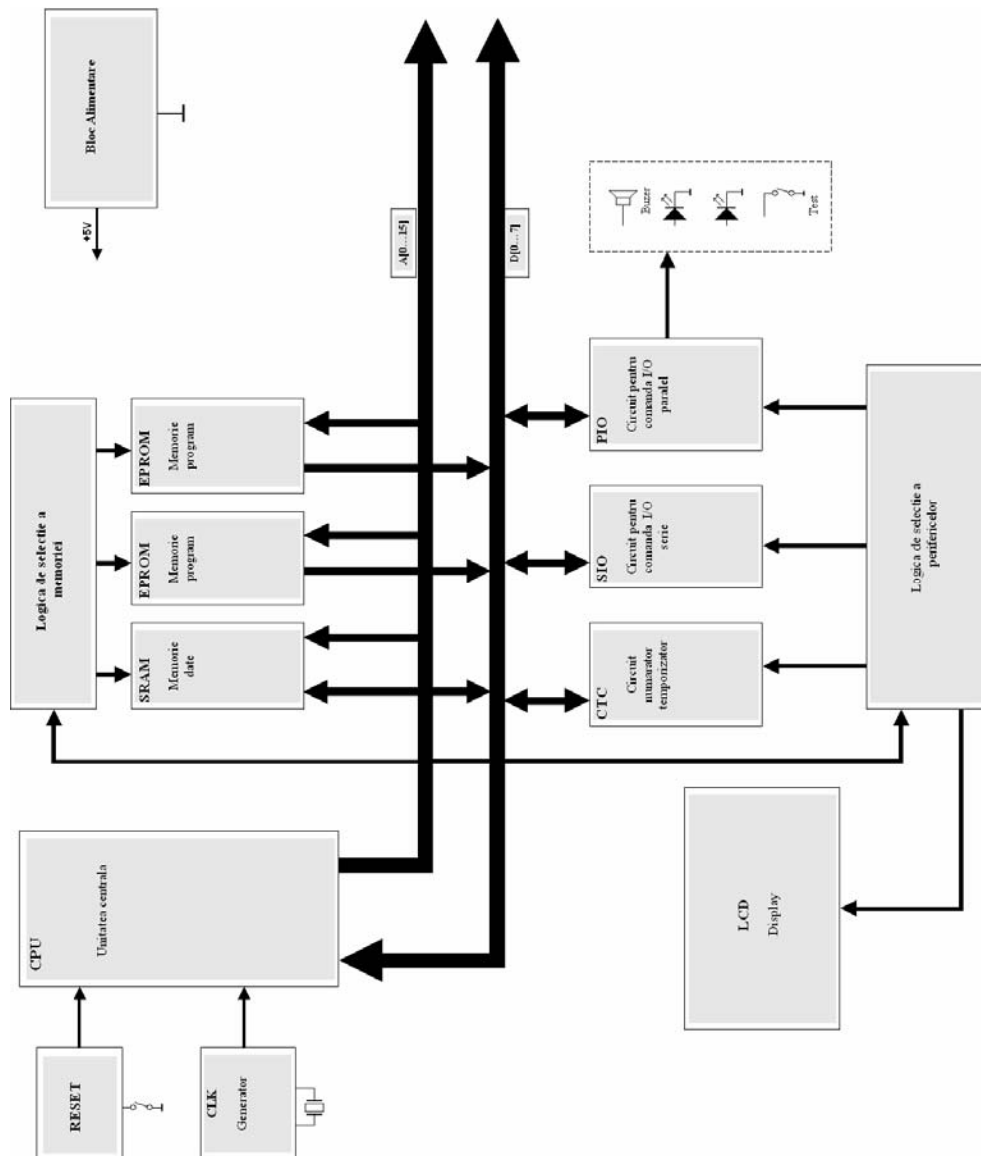
Cu cât mărimea magistralei de date este mai mare, cu atât puterea de calcul a procesorului este mai mare. Magistrala de date este comună tuturor resurselor sistemului. Datorită acestui lucru, circuitele conectate la magistrală trebuie să posede etaje de ieșire de tipul tristate (TS). Printr-o logică de activare și selecție trebuie luate măsuri pentru că în cazul depunerilor de informații pe magistrala de date să se activeze un singur circuit. Altfel, dacă se activează simultan două sau mai multe circuite, se pot produce scurtcircuite și datele sunt alterate. Selecția se realizează prin intermediul unor semnale din magistrala de control.

Memoria de tip ROM nu primește date din cadrul magistralei de date, ea transmițând doar date către microprocesor. În acest sens, proiectantul microsistemului trebuie să prevadă circuite care să asigure protecția la eventualele încercări accidentale de scriere în memoriile ROM. La memoria de tip RAM, precum și la circuitele I/O, astfel de precauții nu trebuie luate, datele putând fi transmise pe magistrala de date, în raport cu unitatea centrală, în ambele sensuri.

2.2 Structura microsistemului cu microprocesor Z-80

Structura microsistemului cu microprocesor Z-80 este prezentată în schema bloc din figura 1. Unitatea centrală este formată din microprocesorul Z-80. Pe lângă aceasta sunt prezente următoarele elemente: circuite aferente pentru logica de control, logică de reset, memorie EPROM în care este înscris programul MONITOR și programele de tip utilizator, memorie RAM pentru salvarea variabilelor în timpul rulării programelor. Pe macheta de laborator se mai găsesc dispozitive specializate de intrare-ieșire pentru comunicația serială (Z-80 SIO), comunicație paralelă (Z-80 PIO), dispozitiv temporizator/numărător (Z-80 CTC). Pentru asigurarea unei interfețe cu utilizatorul cât mai complete, au mai fost atașate un afișaj LCD, un buzzer, două leduri și un buton acestea fiind de uz general, comandate prin program.

Îndrumar de laborator 1



3.1. Programul MONITOR

Programul monitor permite utilizatorului accesul la resursele hardware ale microsistemului printr-un set simplificat de comenzi. Comenzile descrise în continuare sunt specifice microsistemelor din cadrul laboratorului, realizate în jurul unor microprocesoare Z-80. Astfel, prin aceste comenzi se pot realiza :

- afișarea, la ecranul display-ului, a conținutului unei zone de memorie, între două adrese;
- umplerea unei zone de memorie cu o constantă;
- mutarea unei zone (bloc) de memorie într-o altă zonă de memorie;
- compararea conținutului a două zone de memorie cu semnalizarea diferențelor;
- efectuarea de operații simple (adunare și scădere) în hexazecimal;
- vizualizarea locație cu locație a conținutului memoriei cu posibilitatea modificării individuale;
- afișarea regiștrilor μ P-ului cu posibilitatea modificării conținutului acestora;
- rularea programelor utilizatorilor .

La inițializarea microsistemului, pe ecranul display-ului (la calculatorul PC) apare un mesaj prin care se indică utilizatorului faptul că acesta este gata de lucru. De fapt, are loc în prealabil o autoverificare a integrității hardware și dacă aceasta este trecută cu succes se afișează mesajul menționat. Este afișat în continuare prompter-ul (în cazul nostru, semnul “ > ”), care invită utilizatorul la introducerea comenzilor. O comandă se va putea introduce doar după afișarea prompterului, prin tastare în dreapta acestuia. Totodată se afișează și cursorul (ce indică poziția curentă pe ecranul display-ului) printr-o liniuță de subliniere (underline) ce flash-ează (clipește).

OBSERVAȚIE. Se atrage atenția asupra faptului că toate informațiile introduse de la tastatură și, respectiv, afișate pe display sunt în sistemul de numerație hexazecimal.

Programul care realizează interfața dintre microsistem și utilizator, rulat pe calculatorul PC se numește TERMINAL.EXE. El asigură afișarea și transmiterea caracterelor în codul ASCII pe portul serial. După lansarea în execuție a acestui program trebuie configurat după cum este arătat în figura 2. Activarea comunicației între microsistem și calculator se face prin apăsarea butonului **Conectare**.

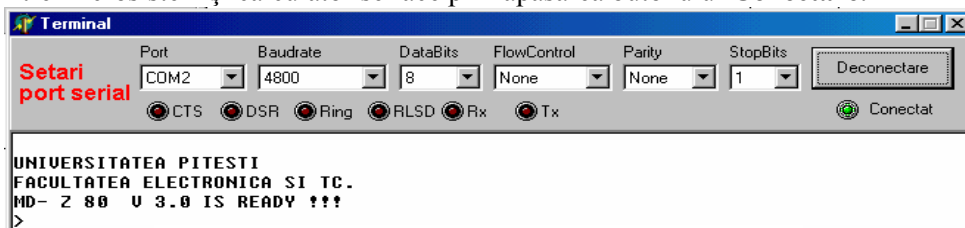


Fig. 2. Configurarea programului Terminal.exe

Setul de comenzi ale programului monitor

a) **DISPLAY** - prezentarea pe display a conținutului memoriei

La prompter se tastează:

> **Dxxxx, yyyy** <CR> sau > **D xxxx yyyy** <ENTER>

unde

xxxx = adresa de început a zonei de memorie ce se vizualizează (în hexazecimal);

yyyy = adresa de sfârșit a aceleiași zone (în hexazecimal); trebuie să avem xxxx < yyyy ;

CR = carriage return.

Ca efect al comenzii, pe ecranul display-ului apare conținutul căutat. De fapt, se prezintă un tabel având pe prima coloană informația de adresă (patru digiți), iar pe următoarele 16 coloane, date, fiecare pe câte doi digiți hexa, separate prin blank.

Mai jos este dat un exemplu în care se face afișarea zonei de memorie de la adresa 100h la 120h

```
>D100 120
0100 61 04 CD 61 04 CD 61 04 3E 11 90 C5 47 80 80 47
0110 CD 61 04 05 C2 10 01 C1 3E 00 B8 C0 23 01 2B 05
0120 C3
>
```

Observații valabile pentru toate comenzile programului monitor:

1. O adresă se introduce printr-un grup de patru digiți în format hexazecimal. Dacă digiții cei mai semnificativi ai informației de adresă sunt "0", ei pot fi omiși. Programul monitor ia în considerare doar ultimii patru digiți introduși. Nu există tasta de ștergere (delete). De fapt, programul monitor nu o tratează. În cazul unei tastări incorecte se reia tastarea până când ultimii patru digiți introduși sunt corecți, fără a face nici o pauză. Existența a patru digiți în format hexazecimal pentru adresă se explică prin mărimea magistralei de adrese la microprocesorul Z-80 (16 biți).

2. Virgula este caracter separator între diferitele câmpuri ale comenzii. În același scop se poate folosi caracterul space (blank).

3. O dată se introduce printr-un grup de doi digiți în format hexazecimal. Dacă digitul cel mai semnificativ al informației de date este "0" el poate fi omis. Programul monitor ia în considerare doar ultimii doi digiți introduși. Ca și pentru informația de adresă, nu există tasta de ștergere. În cazul unei tastări incorecte se reia tastarea până când ultimii doi digiți introduși sunt corecți, fără a se face nici o pauză. Existența a doi digiți în format hexazecimal pentru dată se explică prin mărimea magistralei de date la microprocesorul Z-80 (8 biți).

Îndrumar de laborator 1

4. Practic, orice comandă a programului monitor se termină prin tastarea caracterului terminator (CR sau Enter sau Return). Execuția comenzii introduse este condiționată de această tastare.

5. Comenzile se introduc doar prin folosirea majusculelor (capitals). În acest sens se recomandă apăsarea tastei CAPS LOCK.

b) **FILL** - umplerea unei zone de memorie cu o constantă.

Comanda este posibilă doar atunci când se aplică unei zone de memorie RAM.

>Fxxxx, yyyy, zz <CR>

unde

xxxx = adresa de început a zonei;

yyyy = adresa de sfârșit a zonei; trebuie să avem $xxxx < yyyy$;

zz = data (constanta) cu care se umple zona respectivă.

Dacă se aplică comanda unei zone de memorie ROM, mecanismele de protecție hardware existente în microsistem nu fac posibilă scrierea, comanda neavând nici un efect.

După o execuție a comenzii se afișează un nou prompter, fiind necesară o comandă **DISPLAY** pentru a vedea efectul produs.

c) **MOVE** - mută o zonă de memorie, cuprinsă între două limite, într-o nouă zonă de memorie ce începe cu o adresă specificată.

>Mxxxx, yyyy, zzzz <CR>

unde xxxx = adresa de început a primei zone;

yyyy = adresa de sfârșit a primei zone; trebuie să avem $xxxx < yyyy$;

zzzz = adresa de început a noii zone de memorie; această zonă trebuie să fie una de tip RAM pentru efectuarea cu succes a comenzii; nu este necesară precizarea adresei de sfârșit a acestei zone; zona în care se face mutarea poate fi așezată indiferent de poziția în memoria microsistemului a primei zone. După execuție, se afișează un nou prompter, efectul comenzii putându-se observa prin execuția unei comenzi **DISPLAY**.

d) **COMPARE** - compară o zonă de memorie, cuprinsă între două adrese limită, cu o altă zonă de memorie căreia i se precizează adresa de început, semnalizând eventualele diferențe ce apar între ele.

>Cxxxx, yyyy, zzzz <CR>

unde xxxx = adresa de început a primei zone ;

yyyy = adresa de sfârșit a primei zone; trebuie să avem $xxxx < yyyy$;

zzzz = adresa de început a zonei de memorie cu care se face comparația; nu este necesară precizarea adresei de sfârșit a acestei zone; zona cu care se face comparația poate avea orice poziție în raport cu plasarea în memoria microsistemului, a primei zone.

Dacă zonele comparate sunt identice se afișează un nou prompter. Dacă apar diferențe se prezintă într-un tabel, adresele din prima zonă unde există

Îndrumar de laborator 1

neconcordanțe, precum și cele două date diferite (din prima și, respectiv, a doua zonă).

e) **SUBSTITUTE** - permite vizualizarea și modificarea conținutului memoriei, locație cu locație.

Modificarea este posibilă doar dacă zona de memorie asupra căreia se aplică comanda este una de tip RAM. Vizualizarea este posibilă indiferent de tipul zonei de memorie.

Tastarea acestei comenzi prezintă câteva variante, ea fiind interactivă cu operatorul. Se începe prin tastarea literei S urmată de adresa de la care se dorește vizualizarea și eventual modificarea. După adresă se apasă tasta blank (space). Microsistemul răspunde reafășând adresa introdusă anterior (xxxx), urmată fiind de semnul “:” și de conținutul respectivei locații de memorie (yy). Se afișează apoi o liniuță de despărțire (-). Liniuța are semnificația interogării operatorului asupra modificării respectivului conținut. Dacă acesta dorește modificarea introduce noua dată (zz). Dacă nu, are două variante de continuare la dispoziție. În prima, poate să apese tasta blank (space), trecându-se astfel la adresa imediat următoare (xxxx + 1), cu afișarea conținutului acesteia și lăsând conținutul locației anterioare nemodificat. Lucrând astfel, se va putea vizualiza și modifica conținutul memoriei pas cu pas (locație cu locație). În a doua variantă de continuare, poate să apese tasta CR (enter, return), comanda luând sfârșit și afișându-se un nou prompter.

>Sxxxx xxxx : yy - zz xxxx +1 : uu - vv <CR>

unde

xxxx = adresa locației;

yy, uu = datele memorate în locațiile indicate;

- = liniuța de dialog cu operatorul;

zz, vv = datele modificate;

Dacă nu se dorește modificarea locației se apasă tasta blank. Cu caractere italice (aplicate) este prezentat răspunsul microsistemului.

f) **EXAMINE** - Afișarea și modificarea regiștrilor microprocesorului. Comanda are mai multe variante.

În forma următoare permite afișarea conținutului tuturor regiștrilor microprocesorului :

> X <CR>

Pe linia următoare, la display, apare conținutul tuturor regiștrilor microprocesorului:

>X

A =00 F =00 B =00 C =00 D =00 E =00 H =00 L =00 P =0000 S =FF00

A' =00 F' =00 B' =00 C' =00 D' =00 E' =00 H' =00 L' =00 IX =0000 IY =0000 I =00 R =00

>■

Îndrumar de laborator 1

Într-o a doua formă de tastare a comenzii se poate face și modificarea conținutului registrului dorit. Spre exemplu pentru modificarea conținutului registrului B se tastează :

>XB bb - mm <CR>

unde bb = vechiul conținut al registrului ,
mm = noul conținut (modificat) al registrului

Ca și la comanda S(ubstitute) sunt posibile două variante de continuare. Apăsând blank (space) se trece la vizualizarea și, eventual, modificarea conținutului următorului registru. Dacă modificarea nu se dorește, se apasă blank (space) trecându-se la registrul următor. Într-o a doua variantă, apăsarea tastei CR (return, enter) va provoca sfârșitul comenzii și afișarea unui nou prompter.

g) **GO** - Comandă de rulare a programelor. Comanda se derulează doar dacă anterior în memoria microsistemului s-a depus un program corect, prin folosirea comenzii S(ubstitute).

>Gxxxx <CR>

Este lansat în execuție programul scris de utilizator de la adresa 'xxxx'. Comanda microsistemului trece în totalitate către acest program, cu ieșirea de sub controlul programului monitor.

O altă formă a acestei comenzi este :

>Gxxxx, yyyy <CR>

Această comandă determină rularea programului utilizator de la adresa de început xxxx până la cea de sfârșit yyyy. Pe timpul execuției programului, controlul microsistemului revine programului utilizator. După sfârșitul acestei rulări controlul UC-ului revine programului monitor. Adresa de sfârșit se mai numește adresă de breakpoint (de întrerupere), iar această formă de rulare are denumirea “ **cu breakpoint** “. Este folosită în situații de depanare a programelor, când acestea se rulează pe porțiuni.

h) **HEXADECIMAL** - permite realizarea operațiilor de adunare și scădere între două numere reprezentate în sistemul de numerație hexazecimal.

>Hxxxx, yyyy <CR>

unde xxxx = primul număr (în hexazecimal)
yyyy = al doilea număr (în hexazecimal)

La display, pe rândul următor, se afișează suma și, respectiv, diferența (reprezentate în hexazecimal) celor două numere anterioare și apoi un nou prompter.

Comanda se poate folosi pentru a efectua calcule rapide între numere reprezentate în hexazecimal.

3.2. Mediul de dezvoltare IAR

IAR Systems Embedded Workbench este un mediu integrat flexibil pentru o varietate de aplicații proiectate pe diferite tipuri de procesoare. Este înzestrat cu o interfață grafică ce permite dezvoltarea rapidă a programelor și depanarea lor. Deține următoarele unelte de lucru: Embedded Workbench, C compiler, Assembler, XLINK linker, XLIB librarian, C-SPY debugger.

Crearea unui proiect în IAR Systems Embedded Workbench

Pentru a crea un proiect trebuie urmați următorii pași:

- a) **FILE -> NEW -> PROJECT -> OK**

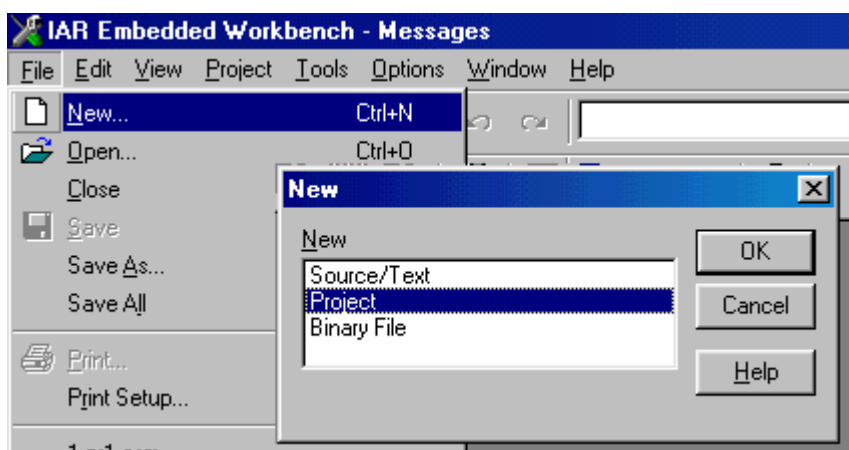


Fig. 3. Comenzile de realizare ale unui proiect

După ce proiectul a fost creat, se deschide o fereastră de editare a programului în samblare prin următoarele comenzi:

- b) **FILE -> NEW -> Source/Text -> OK**

Noul fișier se salvează cu extensia .asm și trebuie plasat în același director în care a fost creat proiectul. În final, va fi adăugat la proiect prin următoarele comenzi:

- c) **PROJECT -> FILES...**

și se va deschide o fereastră cu titlul **PROJECT FILES**. În această fereastră se selectează directorul în care a fost salvat fișierul cu extensia .asm, iar apoi se va da comanda **ADD** pentru a-l adăuga la proiect. Pentru confirmarea operațiilor anterioare este apăsat butonul **DONE**. După aceasta se va observa ca fișierul respectiv va fi adunat la proiectul în care se lucrează.

Îndrumar de laborator 1

Spre exemplu, după ce s-a creat un proiect și s-a adăugat un fișier cu extensia .asm, se introduce următorul program:

1. ORG 0000h
2. LD A,10h
3. LD B,20h
4. LD C,30h
5. END

Liniile 1 și 5 conțin directivele **ORG** și **END**, ale asamblorului, care indică unde va fi plasat codul mașină în memorie și unde se termină asamblarea programului. Liniile 2, 3, și 4 sunt instrucțiuni ale microprocesorului Z80 în limbaj de asamblare.

Pentru a trece acest program în cod mașină trebuie aplicată următoarea comandă:

PROJECT -> MAKE

sau se apasă icoana **MAKE** indicată în figura de mai jos:

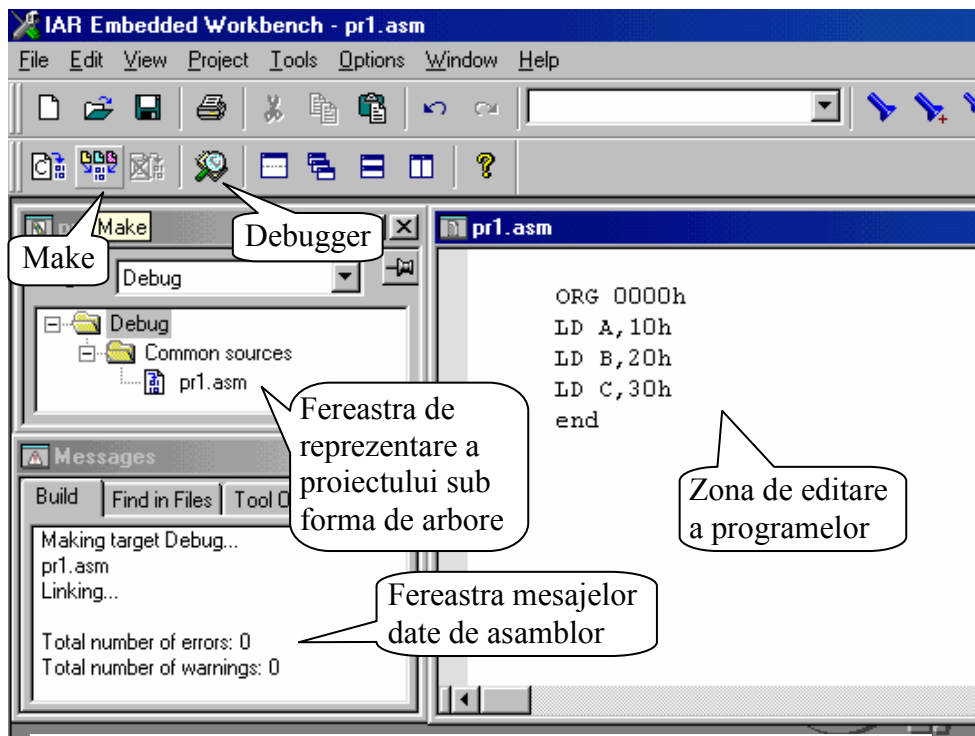


Fig. 4. Modul în care se face editarea și corectarea sintaxei unui

Dacă numărul de erori din fereastra **Messages** este 0, atunci se va activa icoana **Debugger** (Fig. 2) pentru simularea programului editat anterior cu ajutorul aplicației C-SPY debugger (fig.3).

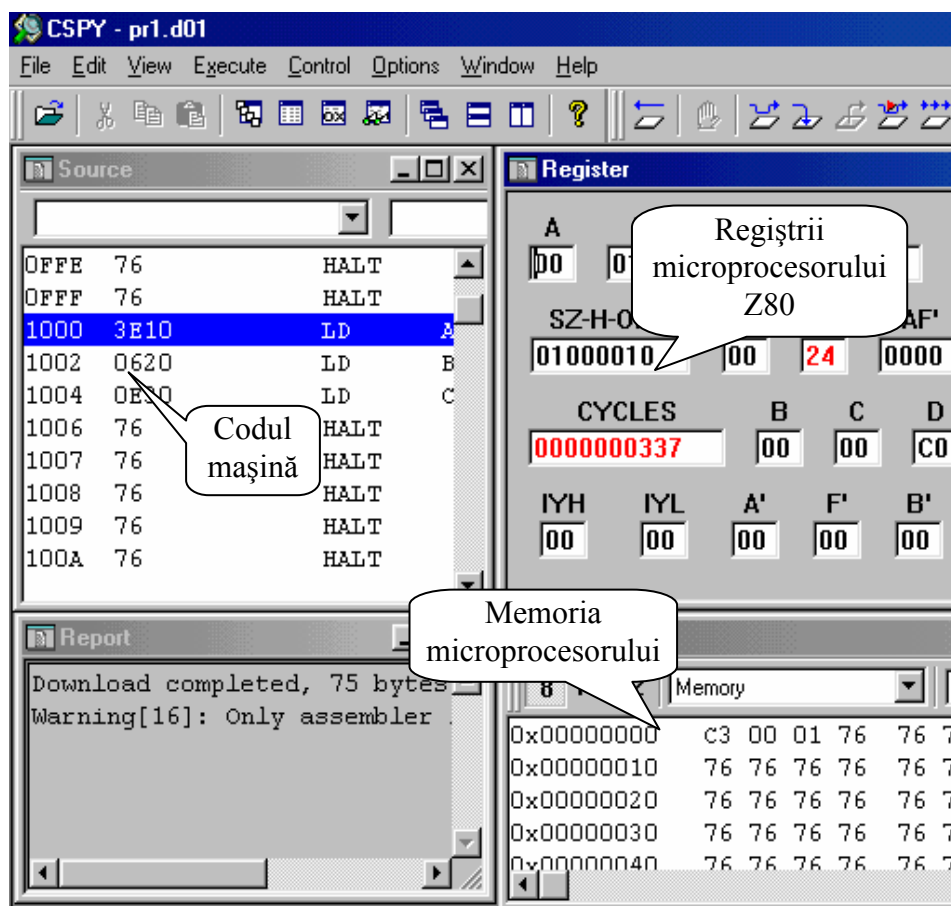


Fig. 5. Modul în care se face depanarea unui program.

Icoanele utilizate pentru rularea programului sunt:



Încarcă registrul PC cu adresa 00h și reinițializează regiștrii procesorului. Simulează resetul procesorului.



Rulează programul pas cu pas fără să intre în subrutine.



Rulează fiecare instrucțiune pas cu pas.

Prin rularea programului pas cu pas se va observa schimbarea regiștrilor afectați prin afișarea acestora cu roșu, la execuția fiecărei instrucțiuni în parte.

Îndrumar de laborator 1

Utilizarea aplicațiilor Embedded Workbench și C compiler dau posibilitatea de editare a programelor în limbajul C. Asamblorul poate genera fișiere cu extensia .hex ce conțin codul mașină a programului în limbajul C sau în limbaj de asamblare. Aceste fișiere se pot descărca în memoria unui microsistem cu microprocesor Z80.

3.3. Sisteme de numerație

3.3.1. Sistemul binar

În sistemele digitale sunt utilizate două nivele de tensiune (uzual 0V și 5V). Prin aceste două nivele de tensiune se materializează cele două valori distincte care au fost alese prin convenție să fie **0** și **1**, corespunzătoare cifrelor utilizate în sistemul de numerație binar.

În microsistemele electronice, unitatea cea mai mică de reprezentare a informației se numește **bit**. Patru biți formează un **NIBBLE**, opt biți formează un **BYTE** (octet), doi octeți formează un cuvânt adică **WORD**, după cum este arătat în figura 6.

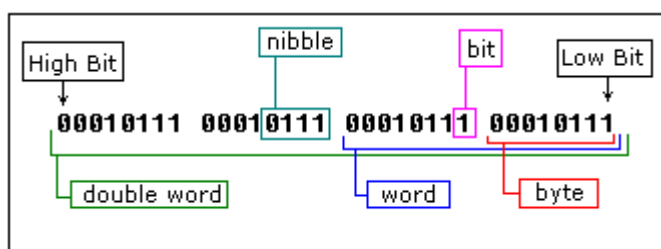


Fig. 6. Moduri de reprezentare a unui număr binar

Transformarea în sistemul zecimal se face prin însumarea produselor dintre respectivele cifre binare cu baza la puterea poziției cifrei. De exemplu, pentru valoarea binară 10100101 avem relația:

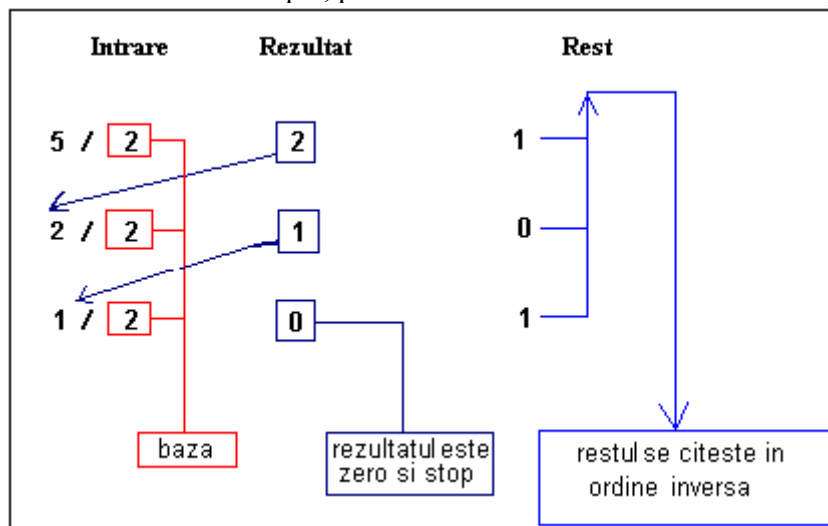
$$\begin{aligned} 10100101_b &= \\ &= 1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 128 + 0 + 32 + 0 + 0 + 4 + 0 + 1 = 165 \quad (\text{valoare zecimală}) \end{aligned}$$

The diagram includes labels: 'baza' (base) pointing to the '2' in the powers of 2, and 'poziție digit' (digit position) pointing to the superscripted powers of 2.

Transformarea din sistemul zecimal în binar se face prin împărțiri succesive la 2 (baza în care trebuie reprezentat). De fiecare dată trebuie reținut rezultatul și lăsat restul. Procesul se continuă până când restul va avea valoarea 0.

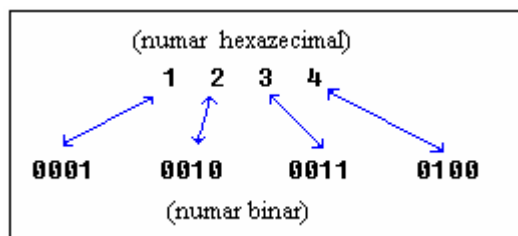
Îndrumar de laborator 1

Resturile rezultate, în ordinea inversă a obținerii, sunt utilizate pentru a reprezenta numărul în baza doi. De exemplu, pentru valoarea zecimală 5 vom avea relația:



3.3.2. Sistemul hexazecimal

Formatul de numerație hexazecimal reprezintă o formă compactă și ușor de citit a sistemului binar. Conversia din sistemul binar în hexazecimal se face ușor, fiecare nibble (4 biți), începând de la bitul cel mai puțin semnificativ fiind convertit într-o cifră hexazecimală după cum este arătat în figura de mai jos.



Transformarea din zecimal în hexazecimal se face asemănător ca și la transformarea din zecimal în binar, cu observația că se schimbă baza. De exemplu, pentru valoarea hexazecimală 1234h avem:

$$1 \cdot 16^3 + 2 \cdot 16^2 + 3 \cdot 16^1 + 4 \cdot 16^0 = 4096 + 512 + 48 + 4 = 4660$$

(valoare zecimală)

Transformarea din sistemul zecimal în sistemul hexazecimal se face prin împărțiri la 16, baza în care trebuie reprezentat respectivul număr. Procedul de calcul este la fel ca și cel de trecere din sistemul binar în sistemul zecimal. De exemplu, pentru valoarea zecimală 39 avem:

Intrare	Rezultat	Rest
39 / 16	2	7
2 / 16	0	2

3.3.3. Reprezentarea numerelor binare cu semn

Este un sistem de codare ce permite reprezentarea numerelor binare cu semn. Poate fi pe un număr oarecare de biți (de obicei, pe un multiplu de 8 biți). Semnul numărului este dat de bitul cel mai semnificativ al reprezentării.

La Z-80 se folosește reprezentarea în complement față de doi (CC2) pe 8 biți. Astfel, CC2 permite reprezentarea numerelor de la $-128_{(10)}$ la $+127_{(10)}$. În funcție de bitul cel mai semnificativ, convenția de semn pentru reprezentare este:

“0” - pentru numere pozitive (inclusiv 0) ;

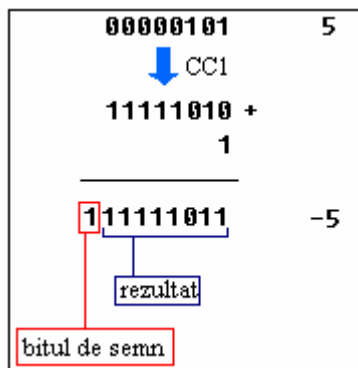
“1” - pentru numere negative .

Regula de transformare a unui număr binar în echivalentul său CC2 este:

- pentru numere pozitive din gama $[0, \dots, 127]$, CC2 este identic cu codul binar;
- pentru numere negative din gama $[-128, \dots, -1]$, CC2 se obține plecând de la codul binar al numărului, care se complementează (se neagă), obținându-se astfel Codul Complement față de 1 (CC1) și la acest număr se adună 1.

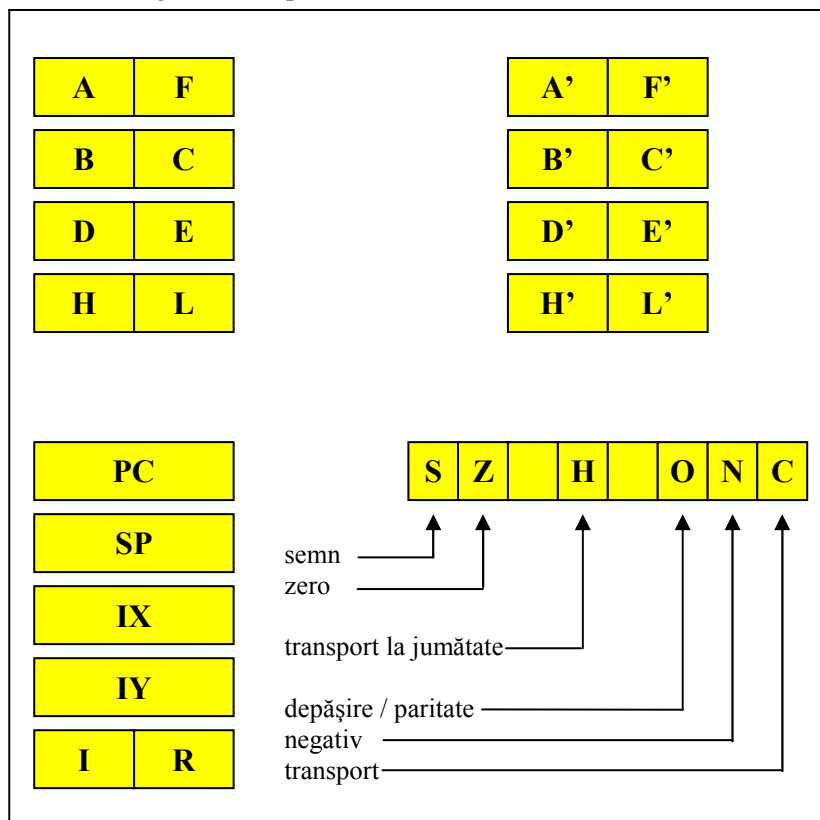
De exemplu, numărul binar cu semn (echivalentul său hexazecimal) +07h are în CC2 reprezentarea 07h (folosind tot sistemul hexazecimal). La acest număr, primul bit (cms) este “0”.

Pentru numărul negativ -05h, se obține CC1 ca fiind 0FAh, iar în CC2, adunând un “1” rezultă 0FBh. La acest număr, bitul cms este “1”, indicând un număr negativ.



3.4. Setul de regiștri ai microprocesorului Z-80

Microprocesorul Z-80 conține două seturi de regiștri generali pe 8 biți, setul principal și, respectiv, setul secundar (notat cu '), precum și un set de regiștri pe 16 biți. Grafic, regiștrii sunt prezentați sub următoarea formă :



Îndrumar de laborator 1

Regiștrii microprocesorului se constituie într-o zonă de memorie RAM, ce poate fi accesată foarte rapid de procesor. Conținutul unui registru rămâne nemodificat până la următoarea scriere care se efectuează în acesta. Regiștrii pe 16 biți sunt permanent accesibili utilizatorului. La un moment dat, utilizatorul are acces doar la regiștrii pe 8 biți aflați în setul principal (respectivul set de regiștri este activ). Datele stocate în setul secundar de regiștri sunt păstrate, dar ele sunt inaccesibile utilizatorului. Aceste informații vor putea fi din nou accesibile, odată cu reactivarea respectivului set de regiștri.

Procesul de reactivare a setului secundar de regiștri este însoțit de dezactivarea setului de regiștri care până în momentul respectiv a fost set principal (acesta devenind astfel set secundar). Revalidarea se face prin două instrucțiuni speciale de tip exchange care vor fi prezentate mai târziu.

a) Regiștrii pe 8 biți

A = acumulator (acumulator) - la nivelul lui se efectuează operațiile aritmetico-logice precum și cele de transfer cu circuitele I/O. De fapt, este cel mai important registru al procesorului, prin el derulându-se majoritatea operațiilor. Se recomandă să nu se folosească acest registru pentru stocarea (memorarea) de informații, altfel putând apare blocaje la efectuarea de operații.

F = flags (indicatori, stegulețe) - registrul indicatorilor de condiții. Conține bistabili corespunzători indicatorilor de condiții din unitatea RALU. Sunt folosiți doar 6 din cei 8 bistabili ai circuitului. Acești indicatori sunt testați de instrucțiuni specifice (instrucțiunile pentru testarea indicatorilor de condiții) ce vor fi prezentate ulterior. În general, indicatorii nu pot fi poziționați direct de utilizator. Ei sunt poziționați ca urmare a efectuării operațiilor aritmetico-logice. Multe operații efectuate de procesor lasă neafecți acești indicatori (de exemplu, operațiile de transfer, de salt, etc).

La microprocesorul Z-80, structura registrului F este următoarea:

S	Z	-	H	-	O	N	C
---	---	---	---	---	---	---	---

S - Sign (semn) - indicator ce prezintă semnul rezultatului operației aritmetico-logice; este practic bitul cel mai semnificativ al rezultatului; acest indicator are semnificație doar dacă operanzii implicați în operație sunt exprimați în cod complement față de 2 (CC2), adică au o reprezentare cu semn. În această situație:

S = 1 reprezintă un rezultat negativ,

S = 0 reprezintă un rezultat pozitiv sau "0".

Z - Zero (zero) - indică dacă rezultatul operației aritmetico-logice este nul

Z = 1 reprezintă rezultat nul,

Z = 0 reprezintă rezultat nenul.

H - Half carry (transport la jumătate) - indică apariția unui transport între nybble-ul inferior și cel superior (adică între D₃ și D₄) la un rezultat al unei operații

Îndrumar de laborator 1

aritmetico-logice. Indicatorul este folosit la operațiile de conversie din binar în cod BCD (binary coded decimally).

P/V - Parity/Overflow (paritate / depășire) - indicator cu dublă semnificație; în urma operațiilor logice indicatorul prezintă paritatea rezultatului obținut, iar în urma operațiilor aritmetice este indicată depășirea gamei de reprezentare (pe 8 biți) de către rezultat.

În cazul parității:

$P = 1$, dacă numărul biților aflați în starea logică “1” din rezultat este un număr par sau zero;

$P = 0$, dacă numărul biților aflați în starea logică “0” din rezultat este număr impar.

În cazul depășirii:

$V = 1$, dacă s-a produs depășirea gamei de reprezentare;

$V = 0$, dacă depășirea gamei nu s-a produs.

Indicatorul de depășire are semnificație doar dacă operanzii implicați în operație sunt reprezentați în cod complement față de 2 (CC2). Microprocesorul Z-80 admite o reprezentare a numerelor în CC2 pe 8 biți, adică se lucrează cu numere cu semn în gama $-128, \dots, +127$.

Ecuția logică a indicatorului V este :

$V = T_7 \oplus T_6$ unde T_7 , T_6 sunt cei doi biți de transport mai semnificativi ai rezultatului.

N - (negative) negativ - este un indicator al operațiilor de adunare și scădere. Astfel, dacă:

$N = 0$, operația anterioară a fost de adunare,

$N = 1$, operația anterioară a fost de scădere.

Indicatorul este folosit la conversiile numerelor binare în echivalentul lor BCD.

Cy - (carry) transport - indicatorul de transport (carry) sau împrumut (borrow) pentru operațiile aritmetice. Operațiile logice lasă neafectat acest indicator. Astfel:

$Cy = 0$, indică faptul că operația aritmetică nu a avut transport (sau împrumut) ;

$Cy = 1$, indică faptul că a apărut transport sau împrumut în urma operației aritmetice.

Indicatorul se poziționează indiferent de modalitatea de reprezentare a operanzilor implicați în operație. Se atrage atenția asupra diferenței ce există între semnificația acestui indicator și, respectiv, a indicatorului de depășire.

Indicatorul Cy este singurul care se poate poziționa prin instrucțiuni specifice direct de către utilizator.

Indicatorii S, Z, P/V, Cy se pot testa direct prin instrucțiuni așa cum se va vedea mai târziu. Ceilalți indicatori (H, N) se pot testa doar indirect prin alte proceduri.

Registrii A și F se pot concatena (uni), rezultând un registru pe 16 biți, **AF**, denumit și **PSW** (programm status word) - cuvântul de stare program. În AF (PSW), F reprezintă octetul cel mai puțin semnificativ (ocmps), iar A este octetul cel mai semnificativ (ocms).

Îndrumar de laborator 1

B, C, D, E, H, L - regiștri de uz general pe 8 biți. De obicei, acești regiștri se pot folosi pentru stocări temporare de informații.

Se pot concatena doi câte doi, regiștrii aflați pe aceeași orizontală din tabelul anterior, rezultând regiștri extinși (pereche, dubli) pe 16 biți : **BC, DE, HL**. Denumirea acestor regiștri nu provine din cuvinte deosebite. Excepție fac regiștrii **H** și **L** care provin din cuvintele englezești High (ridicat) și Low (coborât). De obicei, registrul dublu **HL** este folosit pentru adresarea locațiilor de memorie. În acest registru dublu, **H** reprezintă ocms-ul, iar **L** formează ocmps-ul, rezultând o explicație a denumirii acestora.

În regiștrii extinși amintiți anterior, **B, D, H** formează ocms-ul, iar **C, E, L** reprezintă ocmps-ul.

b) Regiștrii pe 16 biți

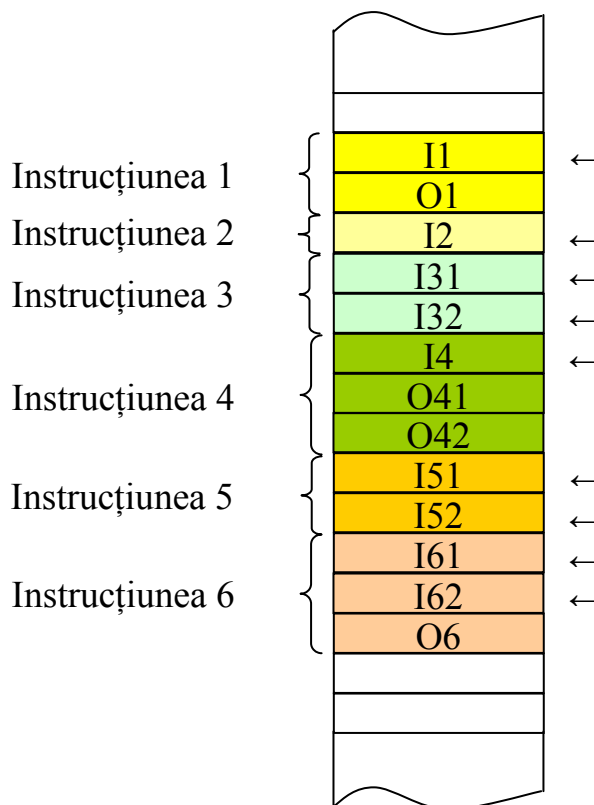
PC = programm counter (numărător de program) - în acest registru procesorul calculează adresa de memorie a următoarei instrucțiuni ce va fi rulată (adresa codului propriu-zis al acestei instrucțiuni).

La microprocesorul Z-80, o instrucțiune poate fi reprezentată în memoria microsistemului pe unu până la patru octeți. Instrucțiunea conține întotdeauna o parte (unu sau doi octeți) ce reprezintă codul propriu-zis al instrucțiunii. Pe de altă parte, poate conține opțional și unul sau doi operanzi. Numărul octeților din codul propriu-zis, precum și numărul octeților de tip operanzi sunt funcție de tipul respectivei instrucțiuni. Octeții de tip cod propriu-zis sunt preluați din memorie (operația fetch), aduși în automatul CROM al microprocesorului și decodificați. Funcție de rezultatul decodificării se dau comenzi pentru execuția instrucțiunii și pentru eventuala preluare din memorie a operanzilor implicați.

Figura următoare exemplifică plasarea în memorie a instrucțiunilor cu cele două componente ale acestora (cod propriu-zis, notat prin litera I și eventuali operanzi, simbolizați prin litera O).

La reprezentarea în memorie a unei instrucțiuni, întotdeauna primul octet (sau primii doi octeți) reprezintă **codul propriu-zis al instrucțiunii** (în figura următoare acest cod este simbolizat prin „←„), iar următorul octet (sau următorii doi octeți) reprezintă **operanzii**.

În registrul PC se calculează adresa locației de memorie la care se află codurile propriu-zise ale instrucțiunii, în vederea extragerii lor din memorie pentru decodificare și execuție.



SP = (Stack Pointer) - indicatorul vârfului stivei. La microprocesorul Z-80, stiva este o parte a memoriei RAM externe. Este folosită în vederea depunerii și extragerii de informații. Informațiile constau de obicei, din datele memorate în regiștrii dubli ai microprocesorului. Folosirea stivei se poate face prin instrucțiuni de către utilizator (manual), dar în anumite situații specifice stiva este folosită automat de mecanismele procesorului, fără intervenția programatorului. Astfel de situații sunt apelul subrutinelor, tratarea cererilor de întrerupere. În aceste situații speciale, în stivă se salvează contextul de lucru al procesorului (PC-ul actual sau locul în care programul aflat curent în rulare este suspendat). După tratarea situației speciale ce a intervenit, tot automat, din stivă se extrage informația depusă anterior, restabilindu-se locul în care a intervenit suspendarea, continuându-se rularea din acest punct.

Stiva este adresată printr-un mecanism de tip LIFO (Last În First Out - ultimul intrat primul ieșit). Registrul SP este folosit pentru a adresa această parte de memorie, indicând de fiecare dată adresa ultimei depuneri (vârful stivei). Inițial, trebuie fixată baza stivei prin încărcarea registrului SP. La o depunere, SP-ul își micșorează valoarea prin operații de decrementare, iar la extragerea din stivă își mărește valoarea prin operații de incrementare.

Îndrumar de laborator 1

Deci, stiva “crește” către înapoi, adică spre adresele de început ale memoriei (către adresa 0000h).

IX, IY = (registrii Index X și Y) . Sunt registre pe 16 biți folosiți în operații de adresare indexată. Procedura de adresare indexată va fi prezentată ulterior.

I = (Interrupt) - registrul de întreruperi. Este un registru pe 8 biți folosit în situațiile de tratare a întreruperilor mascabile, primite de procesor pe intrarea $\overline{INT}$, în modul 2 de tratare a acestora. Modurile de tratare a întreruperilor mascabile vor fi prezentate ulterior. Nu poate fi folosit pentru memorarea altor informații.

R = (Refresh) - registrul de reîmprospătare. Este de fapt un registru pe 8 biți folosit pentru reîmprospătarea eventualelor memorii de tip DRAM conectate la magistralele microprocesorului Z-80. Doar ultimii șapte biți ai acestui registru sunt activi, bitul cel mai semnificativ fiind întotdeauna 0. După execuția oricărei instrucțiuni, conținutul acestui registru se incrementează. În total se pot executa 128 (2^7) cicli de refresh. Conținutul acestui registru se transmite pe magistrala de adrese, pe octetul cel mai puțin semnificativ al acesteia ($A_7, \dots, A_0$) pe timpul efectuării ciclului. Această informație este folosită pentru asigurarea refresh-ului la DRAM-urile conectate extern. În aceste situații, pe octetul cel mai semnificativ al aceleiași magistrale ($A_{15}, \dots, A_8$) se transmite conținutul registrului I, fără nici un fel de însemnătate practică. Această dublă trimitere se constituie într-o explicație a considerării regiștrilor I și R, ca făcând parte din categoria regiștrilor pe 16 biți. Nu poate fi folosit pentru memorarea altor informații.

4. Desfășurarea lucrării

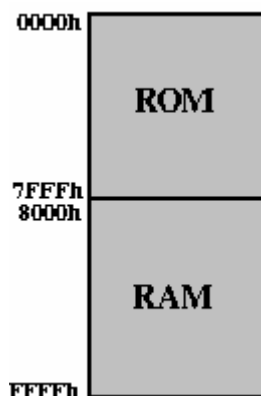
Se va citi și conspecta breviarul teoretic. Se atrage atenția asupra faptului că toate cunoștințele căpătate în acest laborator vor fi necesare și în derularea celorlalte lucrări.

Se vor studia comenzile programului monitor. Se vor efectua operațiile indicate mai jos, respectând ordinea propusă.

OBSERVAȚIE. Din punct de vedere al arhitecturii hardware a microsistemelor aflate în laborator se specifică faptul că zona de RAM (32 Kocteți), se află cuprinsă între adresele 8000h și 0FFFFh. Se recomandă totuși ca utilizatorul să folosească zona cuprinsă între adresele 8010h și 0FF00h. Celelalte locații sunt folosite de programul monitor pentru plasarea de variabile. Zona de memorie ROM (în care se află programul monitor) este cuprinsă între adresele 0000h și 7FFFh (32 Kocteți). Această zonă este protejată la eventualele încercări de scriere accidentală. De la adresa 0000h la 3FFFh se află programul monitor iar de la adresa 4000h la 7FFFh există posibilitatea cuplării unui emulator de memorie ROM, microsistemul dispunând de un soclu adecvat. Prin intermediul unui astfel de aparat va fi posibilă punerea la punct foarte rapidă a programelor. Se poate

Îndrumar de laborator 1

emula o memorie ROM de până la 64Kocteți. Folosind emulatorul, programele vor putea fi scrise în limbaj de asamblare pe un calculator PC. Asamblarea se va face utilizând un program de tip crossasamblor, fișierul obiect rezultat descărcându-se în emulator (download).



1) Se va vizualiza (comanda Display) conținutul zonei de memorie cuprinsă între adresele 8200h și 8300h (**h - reprezintă sistemul de numerație hexazecimal**). Să se identifice câmpurile de adresă precum și de date. Să se specifice numărul de biți pe care este reprezentat fiecare câmp în parte.

2) Se va umple (comanda Fill) zona de memorie cuprinsă între adresele 8200h și 8300h cu constanta 55h apoi se va vizualiza zona de memorie (comanda Display) și se va face remarcă modificării conținutului acestei zone față de situația primei citiri.

3) Se va umple (comanda Fill) zona de memorie cuprinsă între adresele 0200h și 0300h cu constanta 55h apoi se va vizualiza zona de memorie (comanda Display) și se va explica din ce motiv nu este modificată această zonă de memorie.

4) Se va muta (comanda Move) conținutul zonei de memorie cuprinsă între adresele 8200h și 8300h, în zona cuprinsă între adresele 8400h și 8500h.e) și se va vizualiza (comanda Display) conținutul aceleiași zone de memorie cuprinsă între adresele 8400h și 8500h, făcându-se remarcă modificării acestei zone față de situația primei citiri.

5) Se va compara zona de memorie (comanda Compare) cuprinsă între adresele 8200h și 8300h, cu zona de memorie cuprinsă între adresele 8400h și 8500h. Umpleți zona de memorie 8200h 8205h cu constanta 33h și repetați comanda de comparare de la începutul enunțului. Se va stabili dacă există sau nu diferențe între acestea în cele două cazuri.

6) Folosind comanda H se vor aduna și scădea (în hexazecimal) următoarele perechi de numere:

4532h cu 1234h , 0ABCDh cu 9876h , 2AC9h cu 87BFh , 0DF2Bh cu 0E19Ch .

Îndrumar de laborator 1

NOTA. Un program de tip asamblor (crossasamblor) lucrează cu constantele hexazecimale care încep cu o litera (A, B, C, D, E, F) doar dacă în fața acestora se plasează un zero (0). Dacă în astfel de situații nu se introduce cifra 0, asamblorul semnalizează eroare de sintaxă. Pentru introducerea acestor tipuri de date în memoria microsistemului, folosind comenzile programului monitor, cifra 0 poate să nu fie folosită.

7) Cu ajutorul mediului de dezvoltare IAR se va crea un proiect și se introduce programul următor:

```
org 0000h
ld SP,9030h
ld A,41h
buc1a out (0040h),A
      call temp
      call temp
      jp buc1a
org 0020h
temp  ld B,04h
et2   ld C,0ffh
et1   dec C
      jp NZ,et1
      dec B
      jp NZ,et2
      ret
end
```

După ce este compilat proiectul cu comanda MAKE nu rezultă nici o eroare rulați CSPY prin activarea butonului DEBUGGER.

- Identificați în ce zonă se află programul utilizând fereastra SOURCE. Apoi identificați unde este plasat programul în memorie utilizând fereastra MEMORY WINDOW;
- Rulați programul pas cu pas și observați modificările regiștrilor din fereastra REGISTER;
- Observați de câți cicli mașină are nevoie fiecare instrucțiune în parte și câți octeți de cod mașină ocupă.

8) Modificați prima linie de program org 0000h cu 9000h și reasamblați programul. Activați din nou programul CSPY și vizualizați codul mașină a programului din memoria de programe. Scrieți acest cod în memoria microsistemului cu ajutorul programului MONITOR prin comanda Substitute.

- Se verifică introducerea corectă a datelor de mai sus (în ordinea indicată) prin folosirea comenzii Display.
- Se rulează programul prin comanda G9000h (startarea programului se face de la adresa 9000h), remarcându-se efectul acestei comenzi pe ecranul display-ului.

Îndrumar de laborator 1

9) În programul de la punctul 8 se modifică instrucțiunea ld B,04h cu instrucțiunea ld B,03h. Care este efectul asupra programului după rularea acestuia pe microsistem.

10) Care sunt datele echivalente în zecimal pentru următoarele numere exprimate în hexazecimal: 4F5h, 256h, ABCh, 3FF4h.

11) Care sunt datele echivalente în hexazecimal pentru următoarele numere exprimate în zecimal: $675_{(10)}$, $438_{(10)}$, $540_{(10)}$, $810_{(10)}$, $777_{(10)}$.

12) Transformați prin metoda directă din binar în hexazecimal următoarele numere: $01001011_{(2)}$, $11010111_{(2)}$, $10101010_{(2)}$, $10110011_{(2)}$, $11110000_{(2)}$.

13) Efectuați următoarele adunări în sistemul de enumerație hexazecimal: $399h+3A5h$, $300h+F00h$, $ACDh+DD3h$, $E5Bh+25Fh$, $4FCh+978h$.

14) Efectuați următoarele scăderi în sistemul de enumerație hexazecimal: $421h-398h$, $333h-ABFh$, $7CAh-2DE$, $452h-655h$, $1FCh-0FFh$.

15) Calculați complementul față de doi a următoarelor numere exprimate în sistemul de enumerație zecimal: $-34_{(10)}$, $-17_{(10)}$, $-43_{(10)}$, $+2_{(10)}$.